

Table of Contents

Thesis

Background Research

- 1) Approaches to Sentiment
- 2) Data Hierarchy and Source Reliability
- 3) Challenges in Sentiment Extraction
- 4) Advances in Sentiment Modeling
- 5) From Sentiment Measurement to Market Relevance
- 6) Summary of Research

Mathematical Methodology

- 1) Scoring for Textual Events
- 2) Normalisation for Non-Textual Events
- 3) Event-Level Weighting Scheme
- 4) Sub Index Construction
- 5) Composite Index Construction
 - 5.1) Weighting Approaches
 - 5.2) Regime Adaptation
- 6) Interpretability and Attribution

[7\) Implementation Considerations](#)

Thesis

This paper proposes and validates **a composite index for quantifying equity market sentiment constructed from a structured, multi-channel framework** by decomposing it into four distinct components: **market-based signals, public narrative, influencer activity (analysts and insiders), and macroeconomic context**. Each component is independently modeled using normalized, time-decayed, and source-weighted features before being aggregated into a composite sentiment index. Unlike black-box approaches, the framework is fully interpretable and auditable, enabling direct attribution of sentiment drivers. **The resulting index is evaluated as a statistically significant predictor of short-horizon equity returns, with performance assessed across multiple market regimes.**

Background Research

Quantifying equity market sentiment has long been a challenge due to its inherently latent nature. Unlike observable financial variables such as price or volume, **sentiment reflects the collective beliefs, expectations, and reactions of market participants**, which must be inferred indirectly through a combination of behavioral and informational signals. **A robust understanding of sentiment therefore requires distinguishing between *revealed preferences*—what market participants actually do—and *expressed opinions*—what they say or report.**

1) Approaches to Sentiment

Traditional approaches often attempt to capture sentiment through a single proxy, such as news tone or social media activity. However, this simplification overlooks the fundamentally multi-dimensional nature of sentiment. **A more comprehensive framework separates**

sentiment into distinct channels, each capturing a different aspect of market behavior.

Market-based sentiment reflects the actions of participants through measurable indicators such as order flow imbalance, implied volatility, options skew, and short interest. These signals are considered highly reliable due to their direct connection to capital allocation decisions, although they may sometimes be distorted by hedging or liquidity-driven activity.

Public sentiment captures the broader narrative surrounding a stock, derived from sources such as news articles, earnings calls, blogs, and social media platforms. While rich in information, these sources are inherently noisy and susceptible to duplication, manipulation, and sentiment exaggeration.

A third dimension is provided by analyst sentiment, which reflects the views of professional intermediaries through rating changes, target price revisions, and earnings forecast updates. Although more structured and informed, analyst signals are subject to herding behavior and delays in information incorporation.

~~**Finally, insider or action-based sentiment captures the behavior of corporate insiders** through disclosed transactions such as Forms 3, 4, and 5 filings. These actions are often interpreted as strong signals of internal confidence or concern, although they may also reflect non-informational motives such as diversification or liquidity needs.~~

Taken together, these channels provide a more complete representation of sentiment, reinforcing the need for a structured, multi-source approach rather than a single aggregated measure.

2) Data Hierarchy and Source Reliability

Not all sentiment sources contribute equally to price formation. **A key principle in sentiment modeling is prioritizing data based on its causal proximity to market outcomes.** The most reliable signals originate from direct market activity and official disclosures, including exchange data, options markets, and regulatory filings. These sources provide high-frequency, high-integrity information that reflects real economic decisions.

Secondary sources include licensed news wires and institutional research, which offer curated and professionally interpreted information. While valuable, these sources introduce latency

and potential bias.

At the lowest level are public and social platforms, which capture retail sentiment and emerging narratives but are highly susceptible to noise, duplication, and coordinated manipulation. As a result, these sources require careful filtering and weighting to prevent distortion of aggregate sentiment measures.

3) Challenges in Sentiment Extraction

A major challenge in sentiment analysis lies in preprocessing and data quality. **Financial text is often repetitive, multilingual, and context-dependent, with significant ambiguity in entity references and event timing.** Without proper preprocessing, sentiment signals can be severely distorted.

Key preprocessing steps include deduplication to eliminate repeated content, language detection to ensure appropriate model selection, entity linking to correctly map text to specific securities, and relevance filtering to separate stock-specific information from broader market noise. Time alignment is also critical, as sentiment must be accurately matched to market events and trading sessions.

Additionally, sentiment signals benefit from the inclusion of quality indicators such as source reliability, author credibility, and model confidence, which can be incorporated into downstream weighting schemes.

4) Advances in Sentiment Modeling

Sentiment modeling in finance has evolved from simple lexicon-based approaches to more sophisticated machine learning and deep learning methods. Finance-specific lexicons, such as those developed by Loughran and McDonald, address the limitations of general-purpose dictionaries by accounting for domain-specific terminology.

Classical machine learning models, including TF-IDF with linear classifiers, offer efficient and interpretable solutions for structured text classification tasks. More recently, transformer-based models such as FinBERT have achieved state-of-the-art performance by capturing contextual relationships within financial language.

Beyond document-level sentiment, aspect-based sentiment analysis (ABSA) has gained importance for its ability to isolate sentiment across specific dimensions, such as revenue growth, margins, or guidance. This approach enhances interpretability and enables more precise linkage between sentiment signals and market reactions.

For retail-driven sources, auxiliary techniques such as emotion detection and sarcasm recognition can further refine sentiment estimates, although these remain challenging and are typically used as supplementary signals rather than primary indicators.

5) From Sentiment Measurement to Market Relevance

A critical distinction in sentiment research is between textual accuracy and economic usefulness. High classification accuracy does not necessarily translate into predictive power for financial markets. **As a result, sentiment models must be evaluated not only on traditional NLP metrics but also on their ability to explain and predict market outcomes.**

Empirical validation typically involves event studies, correlation and lead-lag analysis, and predictive modeling against baseline financial indicators. Prior research suggests that sentiment can influence short-term price pressure and trading activity, although these effects may dissipate or reverse over longer horizons.

6) Summary of Research

The existing body of research highlights both the potential and limitations of sentiment as a financial signal. While multiple data sources and modeling techniques are available, the absence of a structured, interpretable framework often limits their practical application. The need for a system that integrates diverse sentiment channels, accounts for source reliability, and maintains transparency in signal construction provides the foundation for the methodology proposed in this paper.

Mathematical Methodology

This section formalizes the proposed framework for quantifying equity market sentiment as a structured, multi-channel system. Rather than collapsing all inputs into a single aggregate prematurely, the framework decomposes sentiment into distinct components, applies consistent scoring and weighting mechanisms, and then recombines them into an interpretable composite index.

All signals—textual and non-textual—are ultimately mapped into a unified sentiment representation S_t , where $S_t \in R$ captures both direction and magnitude of sentiment.

Let overall sentiment at time t_i , denoted S_t , be constructed from four independent sub-indices:

$$S_t = f\left(S_t^{(mkt)}, S_t^{(pub)}, S_t^{(inf)}, S_t^{(mac)}\right)$$

$S_t^{(mkt)}$: Market-based sentiment (market data)

$S_t^{(pub)}$: Public narrative sentiment (textual signals)

$S_t^{(inf)}$: Influencer sentiment (analysts and insiders)

$S_t^{(mac)}$: Macroeconomic context

a) Market-Based Sentiment

Captures *revealed preferences* through trading behavior and positioning:

- Signed trade imbalance
- Order book depth imbalance
- Implied volatility and skew
- Put/call ratios

- Short interest and short-sale volume

These signals are treated as direct expressions of capital allocation and are considered the most proximate to price formation.

b) Public Narrative Sentiment

Captures *expressed opinions* from:

- News articles and press releases
- Earnings call transcripts
- Blogs, forums, and social media

Textual data is processed using NLP models to extract sentiment at both document and aspect levels.

c) Influencer Sentiment

Combines:

- Analyst signals: rating changes, target revisions, earnings estimate updates
- Insider activity: net buying/selling from regulatory filings

This channel reflects informed and capital-backed opinions, weighted by credibility and recency.

d) Macroeconomic Context

Captures broader market conditions:

- Volatility indices (e.g., implied volatility benchmarks)
- Interest rates, liquidity indicators
- Market-wide risk sentiment

This channel provides context that conditions the impact of other sentiment signals.

While the four channels define the structural decomposition of sentiment, each channel ultimately consists of a set of underlying observations or events (e.g., trades, news articles, analyst updates, or macro releases).

To construct a quantitative sentiment measure, these heterogeneous inputs must first be converted into a consistent, event-level representation.

1) Scoring for Textual Events

For each given textual event (e.g. a news article, tweet, earnings sentence) i , sentiment models outputs logits:

$$z = (z_{pos}, z_{neu}, z_{neg})$$

which are converted into probabilities via softmax:

$$P_i(y = k) = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

This give you:

$P_i(positive)$: probability the model assigns to the positive class

$P_i(neutral)$: probability the model assigns to the neutral class

$P_i(negative)$: probability assigned to the negative class

with:

$$P_i(positive) + P_i(neutral) + P_i(negative) = 1$$

By compressing these values into a continuous score:

$$S_i = P_i(\text{positive}) - P_i(\text{negative})$$

you get a single continuous sentiment score that captures both direction and strength of the event's sentiment.

Neutral doesn't appear explicitly because it contributes 0.

By using this formulation we ensure:

- Continuous representation (not binary)
- Symmetry around zero
- Compatibility across sources

2) Normalisation for Non-Textual Events

To ensure comparability across heterogeneous signals, all non-textual inputs are transformed into a standardized, unitless sentiment scale prior to aggregation. Raw market and macroeconomic variables (e.g., trade imbalance, implied volatility, put/call ratios) differ in magnitude, units, and distributional properties, making direct comparison or aggregation infeasible.

The primary normalization method used is z-score standardization:

$$S_i = \frac{x_i - \mu_i}{\sigma_i}$$

where x_i is the raw signal value, and μ_i, σ_i are the mean and standard deviation estimated over a rolling window.

This transformation converts each signal into a measure of relative deviation from its historical baseline, enabling consistent interpretation across features.

To ensure directional consistency with sentiment interpretation, normalized signals are further aligned such that positive values correspond to bullish sentiment and negative values to bearish sentiment. For example, signals such as put/call ratios, which are inversely related to sentiment, are sign-adjusted:

$$S_i = - \frac{x_i - \mu_i}{\sigma_i}$$

For bounded or inherently scaled variables, alternative normalization techniques such as min-max scaling may be applied:

$$S_i = \frac{x_i - x_{min}}{x_{max} - x_i}$$

Following normalization and directional alignment, all signals are mapped into a comparable sentiment space, allowing them to be aggregated consistently alongside textual sentiment scores.

Importantly, normalization standardizes scale but does not control for data volume or signal quality; these factors are addressed separately through the weighting scheme.

3) Event-Level Weighting Scheme

Each event i is assigned a weight w_i to reflect its importance:

$$w_i = w_{src} \cdot w_{rel} \cdot w_{conf} \cdot w_{author} \cdot e^{-\lambda \Delta t_i}$$

where

w_{src} : Source reliability (e.g., filings > news > social)

w_{rel} : Relevance probability (is the content about the target asset?)

w_{conf} : Model confidence

w_{author} : Author credibility (e.g., analyst vs anonymous user)

The source reliability weight w_{src} is assigned based on the type and quality of the data source. A hierarchical prior is used, where primary sources such as regulatory filings and official disclosures are assigned the highest weights, followed by institutional research and licensed news, with social media and informal sources receiving lower weights.

Source Type	Example Weight
Filings / official disclosures	1.0
Analyst / institutional research	0.8–0.9
News (Reuters, Bloomberg)	0.6–0.8
Social media	0.2–0.4

These priors may be further refined using historical performance metrics, such as the predictive power of each source.

The relevance weight w_{rel} is obtained from a relevance classification model, which estimates the probability that a given event pertains to the target asset:

$$w_{rel} = P(event\ is\ relevant\ to\ asset)$$

This ensures that general or loosely related content is appropriately downweighted.

The model confidence weight w_{conf} is derived from the sentiment model's output distribution. A simple implementation uses the maximum predicted class probability, while a more robust approach uses the inverse entropy of the distribution:

$$w_{conf} = 1 - \left(- \sum_k P_i(k) \log P_i(k) \right)$$

where higher confidence (lower entropy) results in a larger weight.

The author credibility weight w_{author} reflects the reliability of the information source at the individual level. This may be determined using rule-based heuristics (e.g., analysts and verified institutional sources receiving higher weights than anonymous users), metadata such as follower counts or affiliations, or historical measures of predictive accuracy.

Finally, temporal relevance is incorporated through an exponential time-decay function:

$$e^{-\lambda \Delta t_i}$$

where Δt_i is the time elapsed since the event and λ is a decay parameter. The time-decay parameter λ is specified via source-dependent half-lives, reflecting the persistence of information across different signal types.

High-frequency sources such as social media and intraday market data are assigned short half-lives (on the order of hours), while more durable sources such as filings and macroeconomic indicators are assigned longer half-lives (on the order of days to weeks).

The decay parameter is computed as:

$$\lambda = \frac{\ln 2}{T_{1/2}}$$

ensuring a consistent and interpretable rate of information decay across sources.

Source Type	Practical Value
Social Media	1 - 3 hours
News / Media	6 - 24 hours
Analyst Reports	2 - 5 days
Filings / Official Disclosures	5 - 10 days
Market Data	60 minutes
Macroeconomic Signals	10-30 days

4) Sub Index Construction

For each channel c , the sentiment sub-index at time t is computed as a weighted average of event-level sentiment scores:

$$S_t^{(c)} = \frac{\sum_{i \in t} w_i S_i}{\sum_{i \in t} w_i}$$

This formulation defines $S_t^{(c)}$ as the weighted expectation of sentiment within channel c over the time window t , where the weights induce an empirical probability distribution over events. The normalization by total weight ensures that the sub-index remains scale-invariant and bounded, preventing distortions due to variations in event volume.

This produces a weighted average sentiment score for each channel over a defined time window (e.g., intraday or daily).

Maintaining separate sub-indices allows:

- Channel-level analysis
- Attribution of sentiment changes
- Robustness against noise in any single source

5) Composite Index Construction

The overall sentiment index is constructed as a weighted combination of the channel-specific sub-indices:

$$S_t = \sum_c a_c S_t^{(c)}$$

where $S_t^{(c)}$ denotes the sentiment sub-index for channel c , and a_c represents the corresponding channel weight, subject to:

$$\sum_c a_c = 1, a_c \geq 0$$

Conceptually, the composite index can be interpreted as a mixture of channel-level sentiment distributions, where each weight a reflects the relative importance assigned to a given information source.

This second layer of weighting operates at the channel level, complementing the event-level weighting scheme introduced previously.

5.1) Weighting Approaches

The selection of channel weights α_c plays a critical role in determining the predictive performance of the composite index.

- **Static weights:**

Channel weights are fixed based on prior assumptions regarding source reliability and

proximity to price formation (e.g., market signals receiving higher weight than public sentiment).

- **Dynamic weights:**

Channel weights are estimated and updated over time using data-driven methods, including:

- Rolling regression frameworks
- Information Coefficient (IC) maximization
- Sharpe ratio optimization

Dynamic weighting allows the model to adapt to changing relationships between sentiment signals and market outcomes.

5.2) Regime Adaptation

In addition to time variation, channel weights may be conditioned on prevailing market regimes. Specifically, the relevance of different sentiment channels can vary across:

- High vs low volatility environments
- Bull vs bear market conditions
- Event-driven vs non-event periods

For example, market-based signals may dominate during high-volatility periods, while narrative-driven sentiment may play a larger role during stable conditions. Incorporating regime-dependent weighting allows the composite index to better capture state-dependent dynamics in sentiment formation.

6) Interpretability and Attribution

A key design objective of the proposed framework is interpretability, ensuring that the resulting sentiment measure is not only predictive but also transparent and diagnostically useful.

The framework enables **channel-level attribution**, allowing changes in the composite sentiment index S_t to be decomposed into contributions from each sub-index $S_t^{(c)}$. This provides direct insight into which sources of information (e.g., market

signals, public narrative, or analyst activity) are driving sentiment dynamics at any point in time.

In addition, the structure supports **temporal analysis**, enabling the evolution of sentiment to be tracked across time at both the channel and aggregate levels. This facilitates the identification of trends, regime shifts, and event-driven sentiment changes.

Finally, the framework incorporates **confidence estimation**, derived from the distribution and magnitude of event-level weights. Techniques such as bootstrap or block-bootstrap methods can be applied to estimate confidence intervals around StS_tSt , particularly in settings with sparse or temporally correlated data. This allows users to distinguish between strong, well-supported signals and those driven by limited or low-quality information.

7) Implementation Considerations

A robust implementation of the framework requires a structured and scalable data pipeline.

At the core is an **event-level data store**, where all inputs are recorded with precise timestamps and associated metadata. This enables accurate time alignment and supports both real-time and historical analysis.

Preprocessing steps such as **deduplication and entity resolution** are critical to ensure data quality. Deduplication prevents repeated or syndicated content from biasing sentiment estimates, while entity resolution ensures that each event is correctly mapped to the relevant asset.

Depending on the use case, aggregation may be performed in either **real-time or batch mode**. Real-time processing is required for intraday or event-driven applications, whereas batch processing is sufficient for lower-frequency analysis.

Finally, a **modular system architecture** is essential to allow for iterative improvements. This enables components such as sentiment models, weighting schemes, or data sources to be updated independently without disrupting the overall framework.

Execution Architecture

The sentiment system is implemented as a multi-stage pipeline architecture with a strict separation between data collection, processing, and serving.

The system consists of three primary components:

1. Data Collection and Computation Layer

A scheduled pipeline orchestrates data ingestion, processing, and score generation. This pipeline operates through a set of independent jobs, each responsible for a specific data source or signal type.

2. Storage Layer

All intermediate and final data is stored in a dual-storage system:

- PostgreSQL for persistent, historical storage
- Redis for low-latency access to the most recent computed state

3. API Layer

A FastAPI service serves precomputed sentiment outputs directly from Redis (primary) or PostgreSQL (fallback), with no computation performed at request time.

This design ensures that **data collection, computation, and serving are decoupled**, improving scalability, reliability, and latency.

1) Data Collection and Computation Layer

Data collection is performed continuously at frequencies matched to the natural update rates of each source. Each signal type is ingested independently, without synchronization across channels.

- Raw signals are written to a centralized datastore (**raw_signals**) as they arrive
- No aggregation or composite sentiment computation is performed at this stage
- Each source operates on its own schedule, maximizing data freshness while respecting API constraints

Source Type	Frequency
Market (price/volume)	Every 15 minutes during trading hours (09:30–16:00 ET)
Market (end-of-day snapshot)	21:15 UTC (post-close)
Macro (VIX + sector ETFs)	Hourly during trading hours, daily otherwise
Narrative (news)	Every 30 minutes, 24/7
Influencer (analyst ratings)	Every 6 hours
Influencer (SEC Form 4)	Hourly during business hours

Sentiment computation is performed at a fixed, unified cadence.

- A **global scoring job runs every 30 minutes (on the half-hour)**
- For each of the 502 tickers, the system:
 1. Reads the most recent available signals from all channels
 2. Computes all four sub-indices
 3. Aggregates them into a composite sentiment score

Each scoring cycle produces a **single, fully populated record** in `sentiment_history`, where all sub-indices share the same timestamp.

This design provides:

- **Temporal alignment across channels**, eliminating timing mismatches between signals
- **Consistent cross-sectional comparability**, which is critical for empirical validation
- **Clean panel data structure** for downstream statistical analysis

The system also produces a single sentiment measure based on a smoothed representation of the composite index. The final sentiment score is computed using an exponential moving average (EMA) applied to the composite signal:

$$S_t^{smoothed} = EMA(S_t)$$

The EMA is parameterized using a **4-hour half-life**, ensuring that:

- Short-lived fluctuations and noise decay within approximately 12 hours
- Sustained shifts in sentiment are incorporated within 1–2 hours

The smoothed sentiment series is stored in `sentiment_history` as `composite_score_smoothed`, alongside the corresponding timestamp and supporting metadata.

This smoothing mechanism serves two primary purposes:

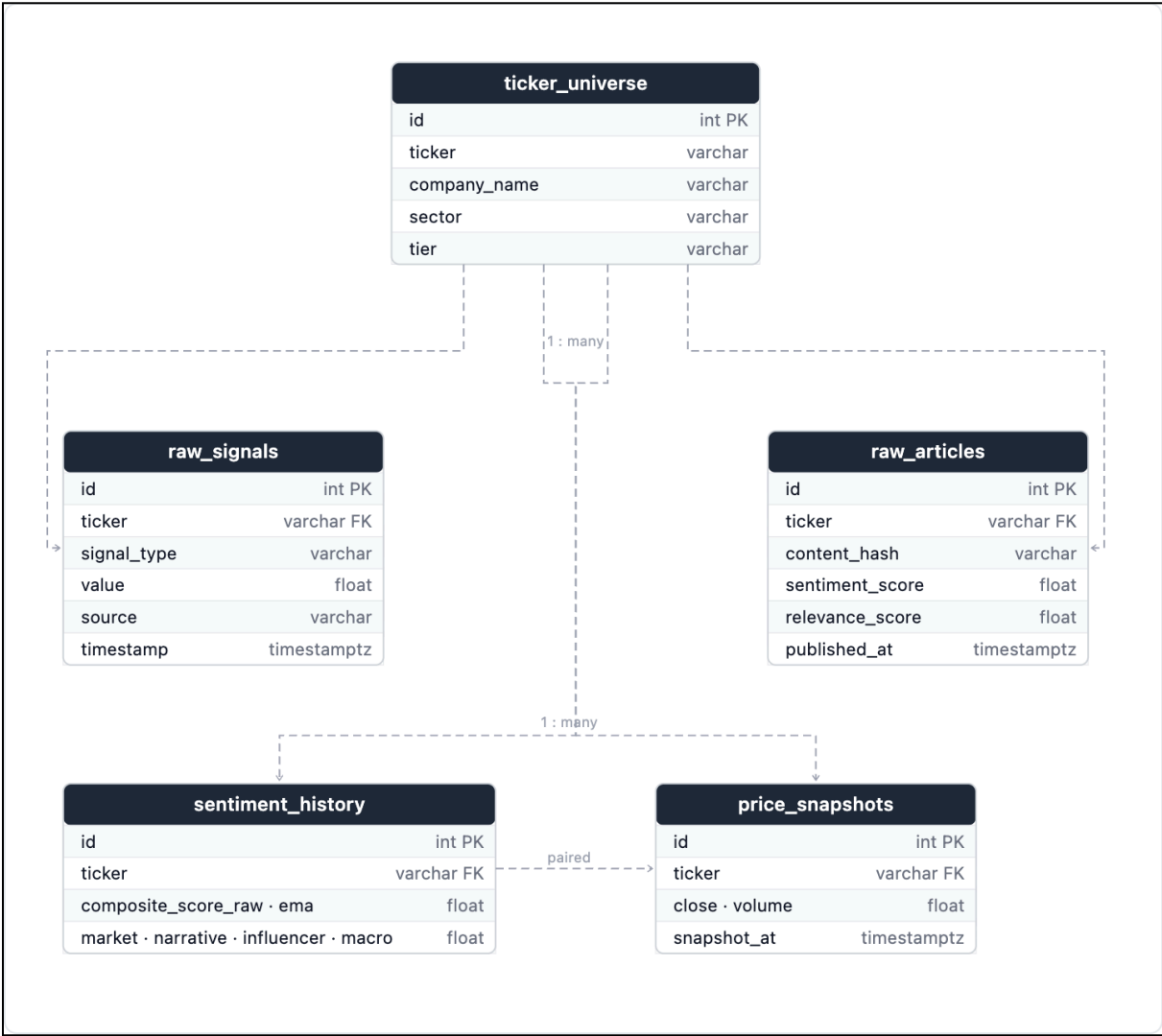
- **Noise reduction**: attenuates high-frequency volatility and transient signal spikes
- **Temporal stability**: produces a more continuous and interpretable sentiment trajectory over time

By construction, the smoothed signal preserves the underlying directional information of the composite index while improving robustness to short-term fluctuations.

2) Storage Layer

The system employs a **dual-storage architecture** designed to separate high-throughput data ingestion from low-latency retrieval, while maintaining a complete historical record of all computed outputs.

The **primary persistent store is a relational database** (PostgreSQL), which serves as the system of record for all raw inputs and computed sentiment outputs.



The database follows an **append-only design** for both raw inputs and computed outputs. Each scoring cycle generates a new row in **sentiment_history**, ensuring full traceability and enabling historical backtesting without recomputation.

No intermediate transformed data is stored. All normalization, weighting, and aggregation steps are performed in memory during the scoring phase, and only final outputs are persisted.

To support low-latency access, the system uses an in-memory key–value store (Redis) as a caching layer. The most recent sentiment state for each asset is stored under a dedicated key (e.g., `sentiment:{ticker}`) with a fixed time-to-live (TTL). This enables sub-millisecond retrieval for real-time applications.

The storage design ensures:

- **separation of raw and computed data**
- **full historical traceability**
- **efficient retrieval of the latest state**
- **independence between ingestion and serving layers**

3) API Layer

The system exposes sentiment outputs through a stateless API layer designed for low-latency, high-availability access.

The API operates exclusively on **precomputed data**, with no real-time computation performed during request handling. This design ensures predictable response times and isolates user-facing performance from upstream data processing.

Upon receiving a request for a given asset, the API follows a read-through retrieval strategy:

1. Attempt to retrieve the latest sentiment state from the in-memory cache (Redis)
2. If unavailable, fall back to the most recent entry in the persistent store (`sentiment_history`)
3. If no data exists, return a structured “no data” response

[insert example]

The response includes:

- the current sentiment score
- channel-level sub-indices
- confidence and diagnostic indicators
- freshness timestamps

- supporting metadata

The API is designed to return the **most recent available state**, even if partially stale. Data quality is communicated explicitly through metadata fields such as timestamps, missing-layer indicators, and confidence scores, rather than withholding responses.

This design provides:

- **low-latency access through caching**
- **decoupling of computation and serving**
- **graceful handling of missing or delayed data**
- **consistent, deterministic responses across requests**

Validation Methodology

The purpose of this section is to evaluate whether the constructed sentiment signal S_t contains predictive information about future returns, and to assess whether the proposed framework components—namely weighting, multi-channel decomposition, and time decay—contribute meaningfully to that predictive performance.

The validation answers five core questions:

1. **Predictive Power:** Does sentiment S_t predict future returns r_{t+h} ?
2. **Signal Quality:** Is the relationship statistically significant and stable?
3. **Economic Value:** Can the signal generate returns in a simple trading strategy?
4. **Robustness:** Does the signal hold across assets, time periods, and market regimes?

This section describes how the mathematical constructs introduced earlier are implemented in practice.

1) Test Universe and Frequency

The validation is conducted on a universe of 502 US-listed equities corresponding to the constituents of the S&P 500 index.

The choice of the S&P 500 ensures:

- high liquidity
- dense information flow
- broad sector representation

2) Data Collection and Sources

Market Based Signals

Market-based signals are collected to capture investor behaviour through observable trading activity. Unlike textual sources, which measure what market participants say, market signals measure what investors actually do through price movement, trading volume, volatility, and positioning. The implemented system prioritises scalable and reliable data collection across the full equity universe.

Data	Signal category	API / Source	Purpose
1. OHLCV (Open, High, Low, Close, and Volume) of 15 minute	Price and volume	yfinance API	Captures core trading activity, price movement, and liquidity
2. 1-period returns	Returns	Computed from OHLCV	Measures directional price pressure across

3. Multi-period returns			multiple horizons
4. Intraday/daily returns			
5. RSI	Momentum	Computed from OHLCV	Captures overbought/oversold conditions and short-term momentum
6. Volume ratio	Volume pressure	Computed from OHLCV	Identifies abnormal trading activity and participation intensity
7. Short interest,	Short positioning	FINRA's REGSHO short volume file	Captures bearish positioning and crowding effects
8. Short-sale volume			
9. Bid-ask pressure	Order flow	Lee-Ready algorithm on OHLCV data	Captures net buying vs selling pressure
10. Bid-ask spread	Liquidity	yfinance API	Measures market depth, execution cost, and confidence

OHLCV data is retrieved using a single batched request across all tickers, significantly reducing latency compared to per-ticker API calls.

Python

```
import yfinance as yf

tickers = ["AAPL", "MSFT", "GOOGL"] # example subset
data = yf.download(tickers, period="2d", interval="1d", group_by="ticker")

# Access OHLCV for a ticker
aapl_data = data["AAPL"]
```

Returns are computed directly from OHLCV data across multiple horizons, enabling the system to capture both immediate price movements and longer-term trends. These include single-period returns, multi-period rolling returns, and intraday or daily return measures depending on the scoring frequency.

Technical indicators such as RSI are computed locally using historical price data stored in the system, eliminating the need for repeated external API calls.

Python

```
import pandas as pd

def compute_rsi(close_prices, window=14):
    delta = close_prices.diff()
    gain = delta.clip(lower=0)
    loss = -delta.clip(upper=0)

    avg_gain = gain.rolling(window).mean()
    avg_loss = loss.rolling(window).mean()

    rs = avg_gain / avg_loss
    return 100 - (100 / (1 + rs))
```

Volume-based signals are derived by comparing current trading volume to historical averages, providing a measure of abnormal activity and participation intensity. This helps identify periods where market conviction is unusually high or low.

Short positioning data is obtained from FINRA's REG SHO short volume file, which provides daily short-sale volume across equities. Short interest and short-sale volume are used to capture bearish positioning and crowding effects, offering insight into the extent to which market participants are betting against a stock.

```
Python

import requests
import pandas as pd
from io import StringIO

def fetch_regsho(date):
    url = f"https://cdn.finra.org/equity/regsho/daily/CNMSshvol{date}.txt"
    return requests.get(url).text

def parse_short_volume(text, ticker):
    df = pd.read_csv(StringIO(text), sep="|")
    row = df[df["Symbol"] == ticker]
    if row.empty:
        return None
    return row["ShortVolume"].values[0] / row["TotalVolume"].values[0]
```

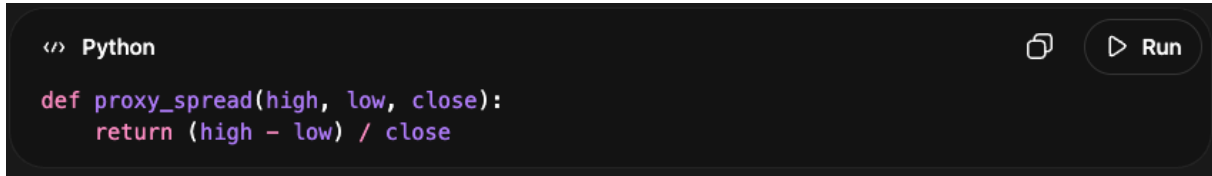
Order flow is approximated using the Lee–Ready algorithm, which infers whether trades are buyer- or seller-initiated based on price changes. This provides a proxy for net buying versus selling pressure in the absence of full order book data.

```
Python

def lee_ready_sign(price_series):
    return price_series.diff().apply(lambda x: 1 if x > 0 else (-1 if x < 0 else 0))

def compute_order_flow(close_prices, volume):
    signs = lee_ready_sign(close_prices)
    return (signs * volume).sum()
```

Liquidity is measured using a proxy for bid-ask spread derived from the high–low price range. While not a direct measure of quoted spreads, this approximation provides a scalable indicator of market depth and execution cost across a large universe of assets.

```
Python  
def proxy_spread(high, low, close):  
    return (high - low) / close
```

This design ensures:

- compatibility with high-frequency update schedules
- independence from strict API rate limits
- efficient scaling across hundreds of assets

By removing unavailable or consistently failing endpoints, the system avoids unnecessary overhead and ensures that only reliable, high-quality signals are incorporated into the market sentiment channel.

Signal construction

Each market signal is constructed as a deterministic transformation of raw market data into a directionally consistent, dimensionless score. The objective is to ensure that every signal, regardless of its underlying unit or scale, can be interpreted on a common bullish-to-bearish axis prior to aggregation. The construction process for each signal proceeds in three stages.

First, raw inputs are retrieved from the relevant data source and validated for completeness, freshness, and integrity. Signals failing validation are excluded from the current scoring cycle rather than imputed, preserving the auditability of the resulting sub-index.

python

```
def validate_raw_signal(value, timestamp, max_age_minutes=30):  
    """Reject signals that are missing, malformed, or stale."""  
    if value is None or not isinstance(value, (int, float)):  
        return False  
    if math.isnan(value) or math.isinf(value):  
        return False  
  
    age_minutes = (datetime.now(timezone.utc) - timestamp).total_seconds() / 60  
    if age_minutes > max_age_minutes:  
        return False  
  
    return True
```

Second, the validated input is passed through a signal-specific transformation function that captures the relevant economic interpretation. Momentum-based signals are computed as log returns over fixed lookback windows (1-day, 5-day, and 20-day), volume signals are expressed as ratios relative to a rolling moving average, and oscillator-based signals such as the Relative Strength Index are computed using the standard 14-period formulation.

python



```
def compute_log_return(prices: pd.Series, horizon: int) -> float:
    """Log return over a fixed lookback window."""
    return float(np.log(prices.iloc[-1] / prices.iloc[-1 - horizon]))

def compute_volume_ratio(volume: pd.Series, window: int = 20) -> float:
    """Current volume relative to its trailing rolling average."""
    avg = volume.iloc[-window:-1].mean()
    return float(volume.iloc[-1] / avg) if avg > 0 else 1.0

def compute_rsi(close: pd.Series, period: int = 14) -> float:
    """Standard 14-period RSI using Wilder's smoothing."""
    delta = close.diff()
    gain = delta.clip(lower=0).ewm(alpha=1/period, adjust=False).mean()
    loss = -delta.clip(upper=0).ewm(alpha=1/period, adjust=False).mean()
    rs = gain / loss.replace(0, np.nan)
    return float((100 - 100 / (1 + rs)).iloc[-1])
```

This staged construction ensures that the resulting signals are economically interpretable, internally consistent, and directly compatible with the downstream normalization and weighting layers. Because each transformation is fully specified, the contribution of any individual signal to the market sub-index can be traced and reproduced from raw inputs alone.

Scoring Pipeline and Observation Frequency

Raw market signals are ingested at a 15-minute cadence during regular trading hours (09:30–16:00 ET), with an additional end-of-day snapshot taken at 21:15 UTC to capture post-close consolidated data. This frequency is selected to balance two competing requirements: sufficient temporal resolution to capture intraday sentiment dynamics, and respect for upstream API rate limits and data licensing constraints. Observations outside trading hours are not generated, since equity market signals are undefined when markets are closed, and the system instead relies on time-decay to attenuate stale values during these periods. Combined with the 60-minute half-life applied to market signals, the practical effect is that the market sub-index attenuates to near-zero weight within several hours of market close. Outside trading hours, the composite sentiment index is therefore dominated by the narrative,

influencer, and macroeconomic channels, which continue to receive fresh observations at their respective cadences.

```
python

from apscheduler.schedulers.asyncio import AsyncIOScheduler
from apscheduler.triggers.cron import CronTrigger

scheduler = AsyncIOScheduler(timezone="UTC")

# Intraday ingestion: every 15 minutes, weekdays, regular trading hours
scheduler.add_job(
    ingest_market_signals,
    CronTrigger(day_of_week="mon-fri",
                hour="13-20", minute="*/15"), # 09:30-16:00 ET in UTC
    id="market_intraday",
)

# End-of-day snapshot: 21:15 UTC, weekdays
scheduler.add_job(
    ingest_market_signals,
    CronTrigger(day_of_week="mon-fri", hour=21, minute=15),
    id="market_eod",
)
```

Each ingestion cycle writes new signal values into the centralized `raw_signals` datastore as immutable records, indexed by ticker, signal type, and timestamp. No transformation, normalization, or aggregation is performed at ingestion time; this separation ensures that the raw observational record remains independent of any downstream methodological choices and can be re-scored under alternative parameterizations without re-ingestion.

python



```
async def write_raw_signals(rows: list[tuple]) -> None:
    """
    Append new signal observations to raw_signals. Immutable: no UPDATE
    or DELETE. Each row is keyed by (ticker, signal_type, timestamp).
    """
    query = """
        INSERT INTO raw_signals
            (ticker, signal_type, value, source, upload_type, timestamp)
        VALUES ($1, $2, $3, $4, $5, $6)
        ON CONFLICT (ticker, signal_type, timestamp) DO NOTHING
    """
    async with db_pool.acquire() as conn:
        await conn.executemany(query, rows)
```

Sub-index computation occurs on a separate, fixed half-hourly schedule, decoupled from the ingestion cycle. At each scoring cycle, the most recent valid observation for each signal is retrieved, transformed, normalized, weighted, and aggregated into the market sub-index. This decoupled design provides three key properties.

1. Scoring is deterministic and reproducible: a given set of raw inputs will always produce the same sub-index value.
2. Scoring frequency can be modified independently of ingestion frequency, supporting both real-time and lower-frequency analytical use cases.
3. Missed or delayed ingestions do not cascade into missing scores, since the scoring layer operates on the most recent available observation rather than on synchronous updates.

Historical Data and Normalisation Baseline

A defensible normalization scheme requires a **stable historical baseline against which current observations can be compared**. Without such a baseline, z-score standardization reduces to a meaningless transformation of short-term noise, and the resulting signals lose their interpretation as deviations from typical market behavior.

The system maintains a **rolling historical window for each signal type**, computed independently per ticker. The window length is defined in observations rather than calendar days, with the size selected to reflect the underlying observation frequency of each signal. Daily-cadence signals — including short volume, analyst metrics, insider activity, and macroeconomic indicators — use a default window of 90 observations, which corresponds to approximately 90 trading days, or roughly one fiscal quarter. Intra-day market signals — including 15-minute returns, RSI, volume ratio, order flow, and bid-ask spread — use a default window of 500 observations, which corresponds to approximately 20 trading days of intra-day bars at the 15-minute scoring cadence.

This per-signal window sizing is selected to balance two considerations:

1. A longer window provides more statistically stable estimates of the mean and standard deviation, reducing sensitivity to short-term volatility regimes; a shorter window adapts more quickly to structural changes in the underlying signal distribution. For daily signals, 90 observations empirically captures a sufficient range of market conditions while remaining responsive to evolving market dynamics. For intra-day signals, 500 observations provides comparable statistical stability without producing baselines that are insensitive to regime shifts. A uniform calendar-day window would oversample intra-day signals — approximately 2,300 observations per 90 calendar days at the 15-minute cadence — and produce a baseline that responds slowly to changing market conditions. For signals with strong cyclical or seasonal components, longer windows of 180 or 252 observations may be applied; for signals with rapidly evolving distributions, shorter windows of 30 or 60 observations may be more appropriate.
2. The historical window is constructed exclusively from observations preceding the current scoring timestamp, ensuring strict point-in-time integrity. No future data is used in the construction of the baseline, eliminating look-ahead bias and preserving the validity of the resulting signals for both backtesting and live deployment. The rolling mean μ_i and standard deviation σ_i for signal i are recomputed at each scoring cycle from the current historical window, allowing the baseline to evolve smoothly as new observations arrive and old ones are retired.

python

```
def normalize_signal(
    current: float,
    history: list[float],
    sigma_floor: float = 1e-4,
    clip: float = 3.0,
) -> float | None:
    """
    Compute a winsorized z-score against a strictly point-in-time history.
    Returns None when the historical standard deviation is degenerate.
    """
    if len(history) < 30:
        return None

    mu = np.mean(history)
    sigma = np.std(history, ddof=1)

    if sigma < sigma_floor:
        return None

    z = (current - mu) / sigma
    return float(np.clip(z, -clip, clip))
```

To prevent degenerate behavior in low-variance regimes, the normalization formula incorporates a small floor on the denominator. When the standard deviation of a signal falls below a minimum threshold, the signal is treated as uninformative for the current cycle and excluded from the sub-index rather than producing artificially large z-scores. Similarly, observations outside a clipped range of ± 3 standard deviations are winsorized to prevent extreme outliers from dominating the resulting sub-index. This combination of point-in-time baselines, rolling re-estimation, and bounded normalization ensures that the market sub-index remains stable, interpretable, and robust to both regime shifts and data anomalies.

Cold Start Handling

Adaptive z-score normalization requires a sufficient historical baseline to produce statistically meaningful estimates. When the available history for a signal falls below the minimum-observation threshold ($n = 30$), the system falls back to a fixed parametric scorer calibrated to the signal's expected distribution. This fallback is applied independently per

signal: an individual signal transitions from parametric scoring to z-score normalization once its history accumulates beyond the threshold, while other signals may continue operating in parametric mode.

Under typical operating conditions, all signals reach the z-score regime within approximately 90 trading days of system initialization for daily signals, and within approximately 20 trading days for intra-day signals. During this accumulation period, the composite sentiment index remains fully populated and interpretable, with parametric scorers providing meaningful sentiment scores derived from the typical operating range of each raw signal.

The parametric scorers used during cold-start are documented in the Appendix. Their formulas are deterministic transformations selected to map each raw signal onto a [0, 100] sentiment scale, with the convention that 50 represents neutral sentiment, values above 50 represent bullish sentiment, and values below 50 represent bearish sentiment. This design ensures continuity in the sub-index output as individual signals transition from parametric to z-score scoring, and preserves the integrity of the steady-state z-score methodology by avoiding the use of imputed or interpolated values during the cold-start period.

In addition to the 30-observation minimum threshold, z-score normalization requires the historical window to be at least 50% filled relative to its target size. This prevents z-score baselines from being computed against trivially short calendar spans during the cold-start period for intra-day signals.

Narrative Signals

Narrative signals are collected to capture the textual discourse surrounding each equity — what market participants and the companies themselves say and write, rather than what they trade. While market-based signals measure revealed preferences through observable actions, narrative signals measure expressed opinions through professional reporting and company-issued disclosures. The implemented system prioritizes sources that produce direct narrative — content explicitly about a target asset — and applies a uniform textual scoring pipeline across all sources.

Data	Signal Category	API/Source	Purpose
------	-----------------	------------	---------

1. Ticker Specific Financial News	Professional	Alpha Vantage	Provides ticker-tagged articles from licensed financial media
2. Real-time company news articles		Finhub	Ticker-keyed endpoint returning full article bodies at higher cadence than AV, hence closing coverage gaps
3. Press releases and material disclosures	Company Issued	SEC EDGAR 8-K filings	Captures the company's own voice on material events such as earnings, guidance changes, M&A, and product launches

Narrative ingestion runs on a 30-minute cadence, 24 hours a day, independent of trading hours. Because public narrative is generated continuously — across global news cycles, after-hours company filings, and weekend retail discussion — the narrative channel is the dominant contributor to the composite sentiment index outside regular market hours, when the market sub-index has decayed under its 60-minute half-life.

Articles from all three sources are written to raw_articles with full article body, source identifier, content hash, publication timestamp, and ticker association.

Signal Construction

Narrative signals are constructed in three stages, applied uniformly across the three textual sources.

Stage 1: Before any article is scored, it must pass two filters: validation and deduplication. Deduplication matters enormously for narrative data because financial news is heavily syndicated. A single Reuters story about Apple's earnings can appear within minutes under Yahoo Finance, MSN Money, Seeking Alpha, MarketWatch, and a dozen regional outlets — often word-for-word, sometimes with minor edits. If the same story is scored five separate times, the sub-index will reflect five votes for what is structurally a single piece of information, distorting the weighted average and overstating the sentiment shift.

The system addresses this in two passes.

Validation is the first of the two filters. Each incoming article is checked for three minimum criteria before any further processing: a non-empty body field (articles consisting only of a title or stub are excluded), a valid publication timestamp (articles with malformed or future-dated timestamps are excluded), and an identifiable source attribution (articles arriving without a recognizable provider tag are excluded).

Articles failing any of these checks are dropped at ingestion and do not enter the `raw_articles` table. This preserves the auditability of the downstream pipeline — every article that reaches scoring has a verifiable provenance and a well-defined point-in-time anchor for the time-decay calculation.

Exact duplicate detection via content hash. When an article arrives, its full body is normalized — whitespace collapsed, case folded — and passed through SHA-256 to produce a 64-character fingerprint. This fingerprint, the `content_hash`, is stored on the `raw_articles` table. Before any new article is written, its hash is compared against recent hashes; an exact match means the body is byte-identical to one already ingested, and the new article is rejected. This catches the cleanest case: syndicators republishing wire stories verbatim.

```
python

import hashlib

def article_hash(body: str) -> str:
    """Stable content hash for exact-duplicate detection."""
    normalized = " ".join(body.lower().split())
    return hashlib.sha256(normalized.encode()).hexdigest()
```

Near-duplicate detection via cosine similarity. Exact-hash matching fails when syndicators make trivial edits: rewriting the headline, adding a publisher tagline, changing a date format. To catch these, each article is converted into a vector of numbers — an *embedding* — that represents the semantic content of the article rather than its literal characters. Two articles with the same meaning produce embeddings that point in nearly the same direction in vector space, even if their words differ.

The system measures this similarity using cosine similarity, which returns a value between 0 (completely different) and 1 (identical direction). Articles whose embedding is more than 85% similar to any article ingested in the recent past are flagged as near-duplicates, and only the earliest-published article in the cluster is retained for scoring. The 85% threshold is calibrated empirically against syndication patterns observed in production: permissive enough to catch reworded wire stories that differ in headline or tagline but identical in substance, and strict enough to preserve genuinely independent reporting on the same event. The threshold was tuned during the implementation phase to produce a deduplication rate consistent with the observed cross-source syndication frequency of approximately 16% of incoming articles.

```
python
from sklearn.metrics.pairwise import cosine_similarity

def is_near_duplicate(
    embedding: np.ndarray,
    recent_embeddings: np.ndarray,
    threshold: float = 0.92,
) -> bool:
    """Flag as near-duplicate if cosine similarity exceeds threshold."""
    if len(recent_embeddings) == 0:
        return False
    sims = cosine_similarity([embedding], recent_embeddings)[0]
    return bool(sims.max() >= threshold)
```

Empirically, this dedup layer removes approximately 16% of incoming articles, with the majority of removals occurring across sources rather than within a single source — confirming that syndication, not intra-source republication, is the dominant duplicate pattern.

Stage 2: Survival of validation and deduplication does not guarantee that an article is meaningfully *about* the focal ticker. Financial news routinely mentions tickers in passing: a sector roundup that lists every major tech company, a macroeconomic article that uses Apple as an illustrative example, a fund manager profile that name-drops their largest holdings. These articles mention the ticker but are not driven by it, and scoring them as direct narrative events for that ticker would inject noise into the sub-index.

The framework distinguishes between **direct narrative** — content whose primary subject is the target asset — and **indirect narrative** — content that mentions the target asset in a broader context. Only direct narrative contributes to the focal ticker's narrative sub-index at full weight.

The three textual sources operationalize this distinction differently:

Alpha Vantage NEWS_SENTIMENT returns a `ticker_sentiment` array on every article, with one entry per ticker the article references. Each entry includes a `relevance_score` between 0 and 1, computed by Alpha Vantage's own relevance model, that quantifies how

central the ticker is to the article's content. A `relevance_score` of 0.9 indicates the article is substantively about the ticker; a score of 0.2 indicates the ticker is one of many mentioned in passing. The system applies a threshold of 0.60: articles meeting this bar are treated as direct narratives for the corresponding ticker, while articles below the threshold are excluded from that ticker's scoring cycle. The same article may exceed the threshold for one ticker and fall below it for another — a story about Apple's supply chain that mentions TSMC in one paragraph might score 0.85 for AAPL and 0.30 for TSM, contributing to AAPL's sub-index but not TSM's.

This use of the provider's `relevance_score` operationalizes the event-level relevance weight w_{rel} defined in section 3 of the methodology without requiring a separately trained relevance classifier.

Finnhub `/company-news` is a ticker-keyed endpoint: the API is queried with a specific ticker, and the response contains only articles already tagged to that ticker by Finnhub. Relevance filtering is therefore performed at the source rather than at scoring time, and these articles enter the pipeline at $w_{rel} = 1.0$ by construction.

SEC EDGAR 8-K filings are filed by the company itself against a specific Central Index Key (CIK). Each filing is unambiguously about exactly one issuer, and these enter the pipeline at $w_{rel} = 1.0$ by construction.

The 0.60 threshold for Alpha Vantage is set as a starting prior. It is tunable — a value too low admits weakly-related articles as noise, a value too high discards genuinely useful coverage — and is a candidate for empirical refinement via Information Coefficient optimization in the validation phase.

Stage 3: Sentiment scoring. Articles surviving validation and relevance filtering are passed through FinBERT (ProsusAI/finbert), a transformer-based sentiment model fine-tuned on financial text. Provider-supplied sentiment scores from Alpha Vantage are not used; running every article through a single documented model ensures methodological consistency and auditable, reproducible scores across all textual sources.

python

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification
import torch

tokenizer = AutoTokenizer.from_pretrained("ProsusAI/finbert")
model = AutoModelForSequenceClassification.from_pretrained("ProsusAI/finbert")

def score_article(text: str) -> tuple[float, float]:
    """Return (sentiment_score, confidence) for one article."""
    inputs = tokenizer(text, truncation=True, max_length=512, return_tensors="pt")
    with torch.no_grad():
        logits = model(**inputs).logits
        probs = torch.softmax(logits, dim=-1).squeeze()
        p_pos, p_neu, p_neg = probs[0].item(), probs[1].item(), probs[2].item()
        sentiment = p_pos - p_neg
        entropy = -sum(p * np.log(p + 1e-12) for p in (p_pos, p_neu, p_neg))
        confidence = 1.0 - entropy / np.log(3)
    return sentiment, confidence
```

Event-level Weighting

After scoring, each narrative event receives an event-level weight that determines its contribution to the sub-index. The weight combines four components from the framework's general weighting formula: source reliability, model confidence, time decay, and (where applicable) author credibility.

1) Source reliability reflects the trustworthiness of the publishing entity, set as a hierarchical prior consistent with the source-reliability framework introduced in section 3 of the methodology. The priors below reflect the principle that filings outrank licensed news, which outranks social aggregation.

Source	Weight
--------	--------

SEC EDGAR 8-K filings	1.00
Alpha Vantage	0.75
Finnhub	0.65

These priors are documented as starting values and are subject to empirical refinement based on each source's predictive performance over time.

2) Model confidence captures how decisive the sentiment model was when scoring an article. FinBERT outputs three probabilities — positive, neutral, negative — that sum to 1. When one class dominates (e.g. $P_{pos} = 0.92$), the model is confident; when the probabilities are spread evenly (e.g., 0.35,0.35,0.30), the model is uncertain.

The system quantifies confidence using the "inverse normalized entropy" of the output distribution where entropy is the measure of uncertainty. (A concentrated distribution has low entropy, a spread distribution has high entropy. Normalizing by the maximum possible entropy ($\ln 3$ for a three-class problem) puts confidence on a 0-to-1 scale, where 1 is maximum confidence and 0 is maximum uncertainty.

A FinBERT output of (0.92, 0.05, 0.03) yields $w_{conf} \approx 0.74$; an output of (0.40, 0.35, 0.25) yields $w_{conf} \approx 0.08$. Confident scores therefore contribute meaningfully more to the sub-index than uncertain ones, even when their raw sentiment values are similar.

3) A half-life is the time required for an article's weight to drop to 50% of its original value. After two half-lives, 25%. After three, 12.5%. This produces a smooth, principled decay rather than an abrupt expiry.

Source	Half-Life	Interpretation
SEC EDGAR 8-K filings	168 hours (7 days)	Material disclosures retain relevance across a full week of trading
Alpha Vantage	12 hours	News cycles attenuate by half within a trading day
Finnhub		

The longer half-life applied to filings reflects their material, durable nature, while news decays on the order of half a trading day, consistent with the empirical persistence of news-driven sentiment.

Aggregation of the Score

The narrative sub-index is recomputed for each ticker at every scoring tick, drawing on all valid narrative events for that ticker — scored articles from the three textual sources. All events share a common downstream representation: each is a (sentiment, weight) pair, where sentiment lies in $[-1, 1]$ and weight is the product of source reliability, confidence, and time decay computed in the previous section.

A volume-shrinkage factor is applied to the raw weighted average to prevent single-event scoring cycles from producing extreme sub-index values. When the number of valid events n for a ticker is small, the sub-index is pulled toward the neutral midpoint via the transformation $S' = 50 + \min(1, n/5) \cdot (S - 50)$, where S is the raw weighted average mapped to the $[0, 100]$ scale. This shrinkage decays linearly toward zero effect as n approaches 5, beyond which the sub-index reflects the unshrunk weighted average. The

intent is to express appropriate epistemic caution when narrative coverage is sparse, rather than allowing a single high-weight article to drive the sub-index to its bounds.

The output is a single scalar in $[-1, 1]$, where positive values indicate bullish narrative consensus, negative values indicate bearish consensus, and values near zero indicate either neutral discourse or counterbalancing sentiment across events. This scalar is written to `sentiment_history` as `narrative_score` at every scoring tick, alongside the market, influencer, and macroeconomic sub-indices.

The narrative sub-index then enters the composite sentiment index at its assigned channel weight of $a_{narrative} = 0.30$ the second-largest weighting after the market channel. This reflects the framework's view that narrative carries genuine predictive signal but sits below market-based signals in causal proximity to price formation — narrative **informs** the trading that drives prices, while market signals **are** the trading. Equally important, the narrative sub-index remains active 24 hours a day, including weekends and overnight hours when the market sub-index has decayed under its 60-minute half-life. During these periods narrative becomes the dominant contributor to the composite, providing continuity in the overall sentiment trajectory and ensuring the system produces meaningful output even when no equity is trading.

Future Additions

Market Index Pipeline

- Add Options data

Narrative Index Pipeline

- **Earnings call transcripts** — aspect-based sentiment analysis on management commentary, separated by topical dimensions such as revenue, margins, guidance, and forward outlook.
- **Direct Reddit and Twitter/X integration** — full textual access to retail forum posts and social media commentary, allowing FinBERT scoring at the post level rather than

relying on third-party aggregated scores. Would supersede the ApeWisdom row once available.

- **Sarcasm and irony detection** — particularly for retail forum data, where literal sentiment classification systematically misreads inverted-tone posts. A known limitation of finance-domain transformer models.
- **Indirect narrative routing** — articles with relevance scores below the direct-narrative threshold but above a lower bound would be routed to a ticker's sub-index at a downweighted contribution, capturing sector and peer-driven narrative spillover.
- **Per-aspect sentiment dimensions** — decomposing the narrative sub-index into thematic axes (financial performance, product, regulatory, leadership) to support finer-grained sentiment attribution and channel-level interpretability.
- **Retail attention signal (ApeWisdom or equivalent)** — non-textual ingestion of mention volume and upvote counts from retail forum aggregators, treated as an *attention* signal rather than a sentiment signal. Useful for capturing crowd attention spikes that may precede price movement, but methodologically distinct from sentiment because the underlying posts are not retrieved or scored. Would feed into a separate sub-signal or be folded into macro context rather than narrative.

Appendix

Figure 1.1 End-to-End Data Flow Diagram

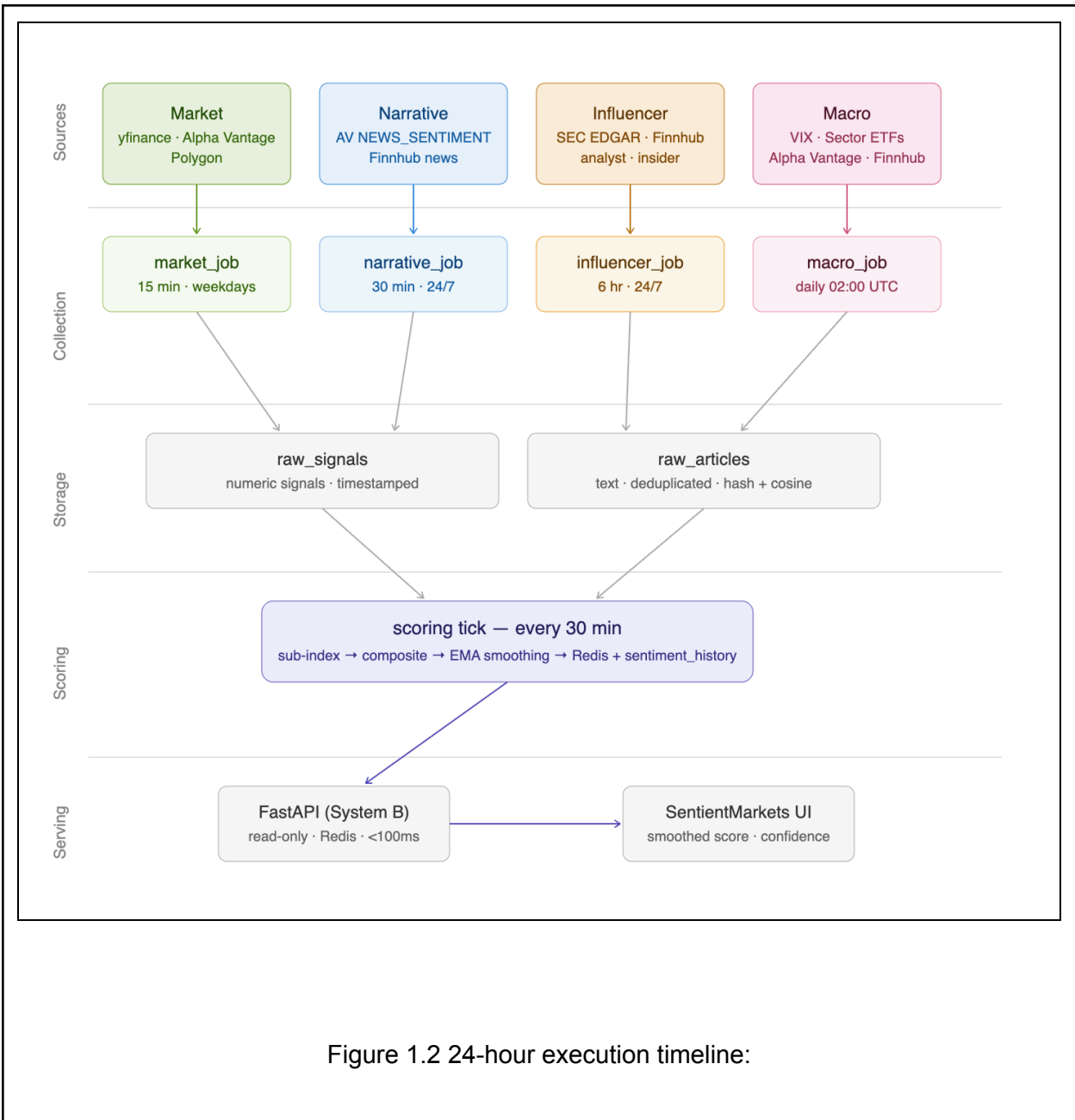


Figure 1.2 24-hour execution timeline:

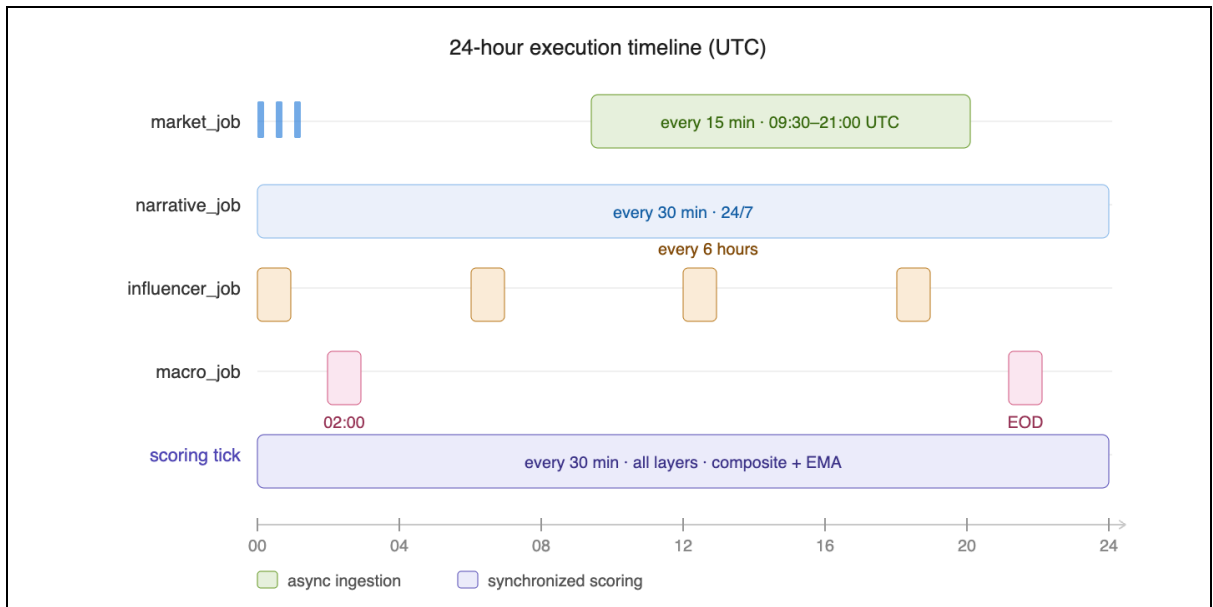


Figure 1.3 Repository Structure

Repository structure — sentimentapi/

```

sentimentapi/
├─ pipeline/                               - System A: ingestion & scoring
│   ├── scheduler.py                     - APScheduler jobs, progress bars
│   ├── orchestrator.py                  - per-ticker score + write cycle
│   ├── rate_limits.py                   - semaphores, guarded_get(), retry
│   ├── sources/                          - external API fetchers
│   │   ├── market.py                   - yfinance · AV · Polygon
│   │   ├── narrative.py                 - AV NEWS_SENTIMENT · Finnhub news
│   │   ├── influencer.py                - SEC EDGAR · Finnhub analyst
│   │   └─ macro.py                      - VIX · sector ETF closes
│   ├── scoring/                         - signal → sub-index math
│   ├── confidence/                      - staleness, confidence score
│   │   └─ staleness.py                  - market-hours-aware freshness
│   ├── persistence/                     - DB + Redis writers
│   ├── explanation/                     - driver extraction, templates
│   └─ nlp/                              - article dedup (hash + cosine)
├─ api/                                   - System B: read-only FastAPI
│   └─ routes/                           - /score · /history · /tickers · /health
├─ db/                                   - connection pools
│   ├── connection.py                   - asyncpg pool (init_pool)
│   └─ redis.py                         - redis-py async (init_redis)
├─ migrations/                          - numbered SQL migrations
├─ backfill/                             - one-time OHLCV + RSI seed
├─ tools/                                - db_viewer · export · key gen
├─ tests/                                - 95 tests (pytest)
├─ main.py                               - startup: pool → redis → scheduler
├─ railway.toml                           - Railway deploy config
└─ requirements.txt

```

Figure 1.4 Parametric Fallback Formulas

Signal	Parametric formula
rsi_14d	$50 - 1.25 \times (\text{rsi} - 50)$, clipped to $[0, 100]$

return_1d	$50 + 50 \times \tanh(\text{ret} / 0.02)$
return_5d	$50 + 50 \times \tanh(\text{ret} / 0.05)$
return_20d	$50 + 50 \times \tanh(\text{ret} / 0.10)$
volume_ratio	$\text{clip}(50 + 15 \times \tanh(\text{vr} - 1), 20, 80)$
order_flow_imbalance	$50 + 50 \times \text{clv}$
bid_ask_spread_bps	$50 - 50 \times \tanh((\text{bps} - 10) / 30)$
insider_net_shares	$50 + 50 \times \tanh(\text{shares} / 100,000)$
analyst_buy_pct	$25 + 50 \times \text{pct}$
vix	$50 - ((\text{vix} - 22) / 8) \times 25, \text{ clipped to } [0, 100]$